

IWS, WS 25/26

12.02.2026

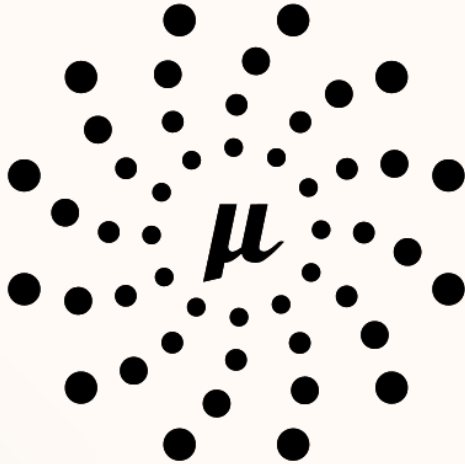
Florian Babel (he/him)

Abdulrahman Dabbagh (he/him)

Cyrille Raven Ash Zèro Yuki Falkenstein (none/they/them)

Falko Piwinger (he/him)

Julia Sugakov (she/her)

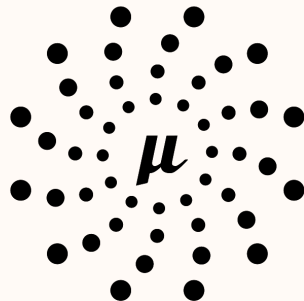
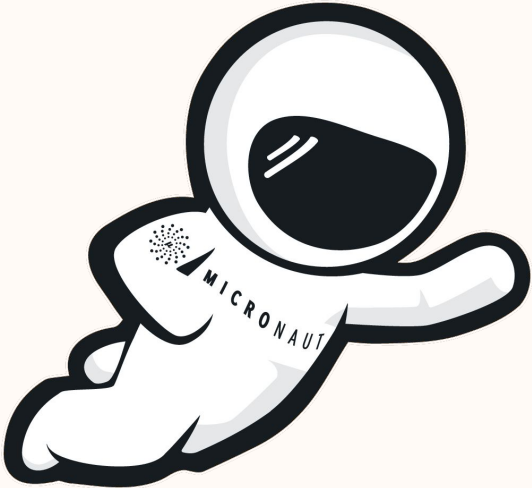


M I C R O N A U T[®]

Startprotokoll

- | | | | |
|---|-------------------------|-----|--------------------------------|
| 1 | Einführung & Grundlagen | 5 | Validation & Errorhandling |
| 2 | Setup & Endpunkte | 6 | Testen & OpenAPI |
| 3 | Dependency-Injection | (7) | (Konfiguration & Datenzugriff) |
| 4 | REST Clients & AOP | 8 | GraalVM & Abschluss |

 Mittagspause (45min)



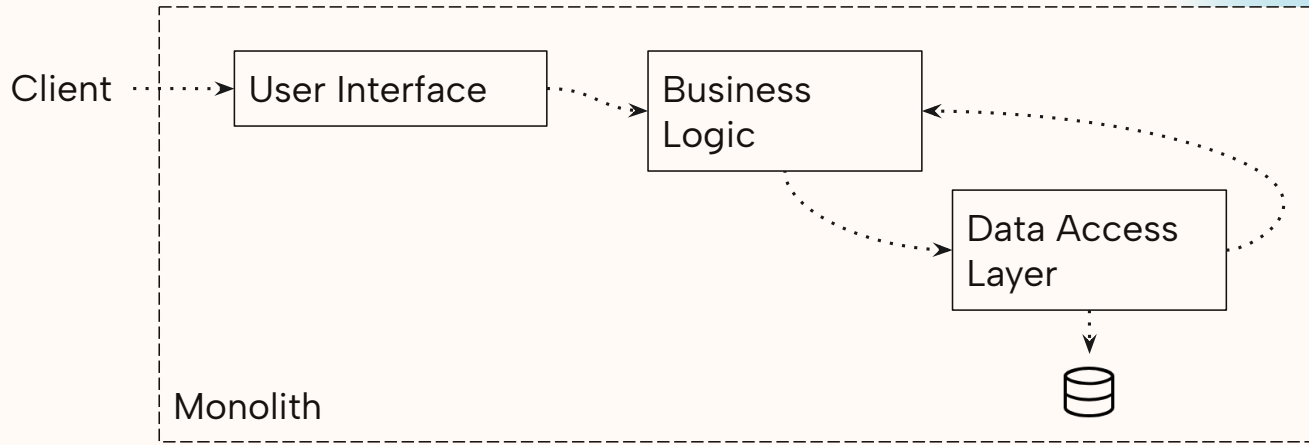
M I C R O N A U T[®]

"A modern, JVM-based, full-stack framework for building modular, easily testable **microservice** and **serverless** applications"

Monolith

eine zusammenhängende Anwendung, meist mit einer Datenbank

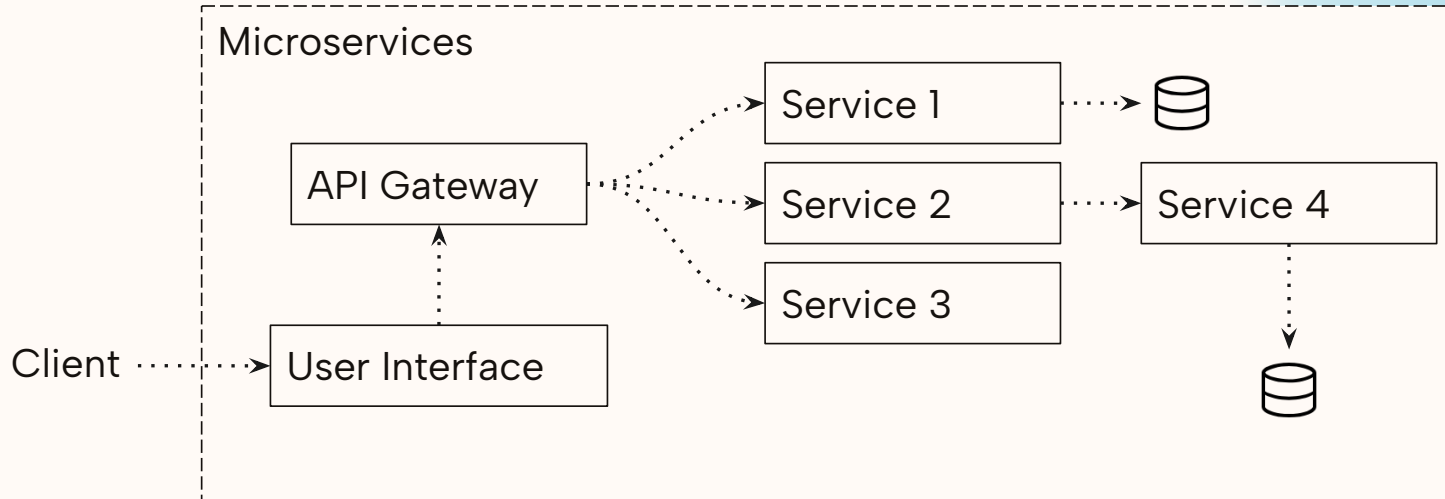
Problem: enge Kopplung zwischen Funktionen



Buzzword – Microservices

Anwendungsfälle sind voneinander getrennt, oft mit eigener Datenbank

Vorteil: Funktionen sind voneinander entkoppelt



Buzzword – Microservices

Wartbarkeit

kleiner, gut
verständlicher
Kontext

Flexible Skalierung

Services skalieren
und deployen
voneinander
unabhängig

Technologische Freiheit

Das beste Tool für
ein spezifisches
Problem

Wieder- verwendbarkeit

Services als
Bausteine für
andere Services

Ausfallsicherheit

Ausfall einzelner
Dienste, statt
kompletter
Systemausfall



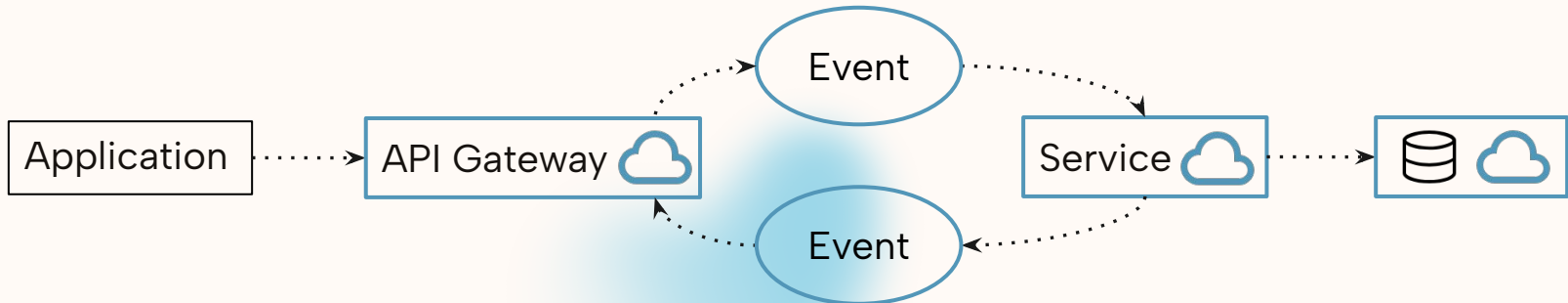
Buzzword – serverless

event-driven architecture, Leistung wird nur nach Bedarf abgerufen

Cloud Anbieter übernimmt Bereitstellung, Skalierung und Wartung der Server

Function as a Service (FaaS): “just run this code”
Serverless Database: “just store / give me this data”

- > **Serverless**
- > **Look inside**
- > **Servers**



Buzzword – Cloud-Native



Buzzword – Cloud-Native

Distributed Configuration

Zentrale
Verwaltung von
Microservice
Konfiguration

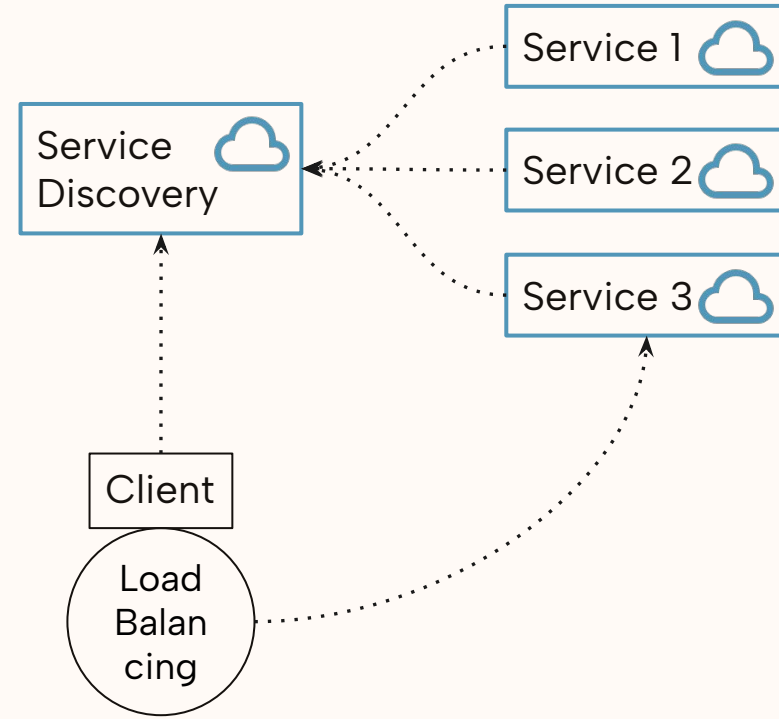
Automatic Cloud Platform Detection

Client Side Load Balancing

Client wählt
Server
selbstständig

Service Discovery

Services können
sich gegenseitig
finden



APPOLLO 67

beyond the cloud...



APOLLO 67

Ihr seid Space Pilots eures Planeten!

Ihr wollt eine florierende Wirtschaft eures Planeten und viele Credits verdienen.

Die **Hightech Technologie** "Micronaut Team" schickt über einen Satelliten Missionen für die ihr bezahlt werdet.

Leider ist "Micronaut Team" noch unvollständig, weil die Entwicklung etwas zu geheim war.

Eure Aufgabe: "Micronaut Team" in Betrieb nehmen



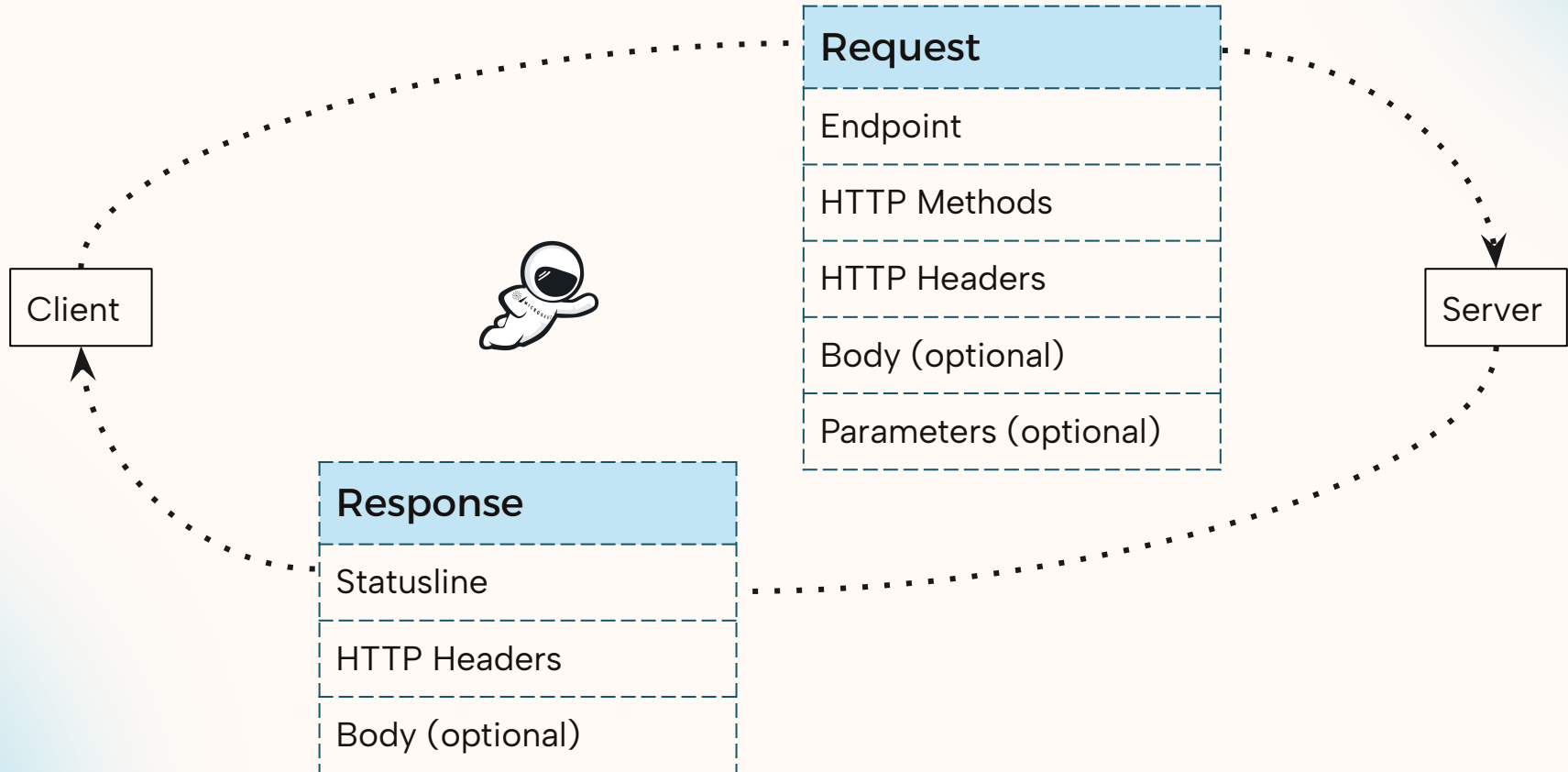
Aufgabe 1 - Welcome to Space Sector 4-Apfel-07

- Findet euch in Zweiergruppen zusammen, idealerweise sollte mindestens eine Person Erfahrung in Java haben.
- Öffnet IntelliJ und pullt das vorinstallierte Repository (Ctrl + T).
- Wechselt anschließend auf den Branch 0-vorher (git checkout 0-vorher && git pull).
- Navigiert zu **src/main/resources/application.yml** und tragt euren eindeutigen Token in folgendem Format ein **<teamnummer>-<erste Buchstaben der Vornamen>-<ein zufälliges Wort>** (am Anfang der Datei). Wählt zusätzlich einen Namen für euer Raumschiff (name) und euren Planeten (planet-name).
- An der Tafel findet ihr die Base-URL, welche ihr hier eintragt:
- Startet den Service

RESTful

Space Missions Start On The Ground

HTTP Requests und Responses



RESTful Designprinzipien

Uniform Interface

gleiche Anfrage
für dieselbe
Ressource

Statelessness

jede Anfrage
muss alle nötigen
Informationen
senden

Client-Server Decoupling

Client und Server
sind vollständig
getrennt

Cacheability

Antworten
können auf Client
zwischen-
gespeichert
werden

Layered System Architecture

Anfragen und
Antworten
können über
autorisierte
Vermittler geleitet
werden

Code On Demand (opt)

Server können
Client-Funktionali-
täten
vorübergehend
erweitern oder
anpassen

CURL

- curl ist ein plattformübergreifendes command-line tool
- Aber auch eine Bibliothek (libcurl, die Engine von curl)
- Mit curl können Daten über URLs zwischen Servern und Clients übertragen werden
- curl unterstützt Protokolle wie HTTP(S), IMAP, (S)FTP, POP3, ...
- "curl is used daily by virtually every Internet-using human on the globe"
- Wir benutzen heute curl mit HTTP zum manuellen testen



CURL – Basics

GET	<code>curl https://api.example.com/users</code>
POST	<code>curl -X POST https://api.example.com/users \ -H "Content-Type: application/json" \ -d '{"name":"Sally","email":"sally@micronaut.io"}'</code>
PUT	<code>curl -X PUT https://api.example.com/users/123 \ -H "Content-Type: application/json" \ -d '{"name":"New Name","email":"new@email.com"}'</code>
PATCH	<code>curl -X PATCH https://api.example.com/users/123 \ -H "Content-Type: application/json" \ -d '{"email":"updated@email.com"}'</code>
DELETE	<code>curl -X DELETE https://api.example.com/users/123</code>

Flag	Beschr eibung
-X	HTTP Methode anpassen
-d	Daten bzw Body anpassen
-H	Header anpassen

Basics

Everyone Starts Somewhere

How to set up a Server

Erstellen eines Projekts mit Micronaut Launcher:

<https://micronaut.io/launch/>

Erzeugt Projektstruktur und "Application" Klasse

```
import io.micronaut.runtime.Micronaut;

public class Application {

    public static void main(String[] args) {
        Micronaut.run(Application.class);
    }
}
```

Intermission: Annotations

@Controller("Endpunkt")

Methoden für @Get/@Post/@Put

Standardmäßig Content-Type json

Alternativ **@Produces(MediaType.TEXT_PLAIN)**

Ihr werdet viele Annotationen sehen!

Details zu Annotationen später

```
import io.micronaut.http.MediaType;
import io.micronaut.http.annotation.Controller;
import io.micronaut.http.annotation.Get;

@Controller("/hello") // (1)
public class HelloController {

    @Get(produces = MediaType.TEXT_PLAIN) // (2)
    public String index() {
        return "Hello World"; // (3)
    }
}
```

Intermission: Annotations

@Body datentyp <body>

```
Controller("/hello")
public class HelloController {

    @Get
    @Produces(MediaType.TEXT_PLAIN)
    public String index() {
        return "Hello World";
    }

    @Post
    @Produces(MediaType.TEXT_PLAIN)
    public String postHello(@Body String name) {
        return "Hello " + name + ", I received your post!";
    }
}
```

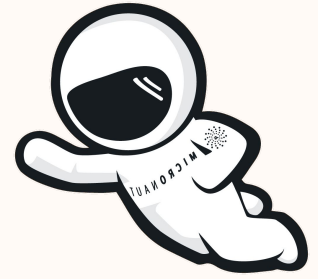
Aufgabe 2 – Cadet, we want to explore!

Wir senden euch über unseren Satelliten mögliche Orte mit habitablen Planeten zu, die ihr euch mal anschauen sollt.

Wir benötigen Schnittstellen, damit unser Satellit mit euch kommunizieren kann. Durch Störsignale bekommt ihr manchmal fehlerhafte Missionen mit Belohnungen kleiner gleich Null, die ihr bitte unbedingt ablehnt!

Implementiert die Endpunkte **GET /missions/** und **POST /missions/complete**.

Genaue Anweisungen findet ihr unter [Aufgaben/Aufgabe_02.md](#)



Dependency Injection

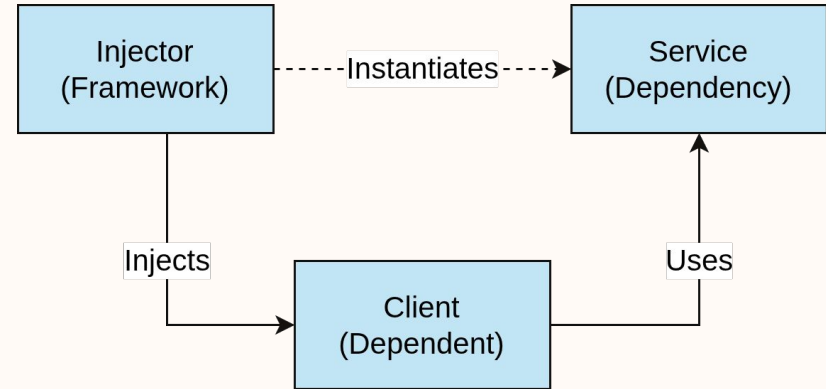
Motivation

- Was passiert wenn wir ein anderes MissionLogRepository benutzen möchten?
- MissionService muss geändert werden 😞
- Wie können wir das umgehen?

```
public MissionService() {  
    this.repo = new InMemoryMissionLogRepository();  
    this.rng = new MissionRng();  
}
```


Dependency Injection

- Jede Klasse gibt an welche Dependencies sie benötigt
- Framework injiziert Beans in Klassen
- Micronaut nutzt den Jakarta-Inject Standard (JSR-330)



Bean vs Bean

JavaBean (Java SE)

- Altes Java Konzept
- Wiederverwendbarkeit von Klassen
- Anforderungen
 - Leerer Constructor
 - private Felder mit Getter und Setter
- Implementiert Serializable

Jakarta Managed Bean (Jakarta EE)

- Annotiert mit Scope-Annotation
- Framework kümmert sich um den Lebenszyklus und Abhängigkeiten der Instanz
 - Framework initialisiert Objekte und injiziert diese in andere Beans
- Wird normalerweise nicht mit *new* genutzt



Klasse zur Bean machen

- Scope Annotation an der Klasse
- Das Framework ist jetzt für den **Lebenszyklus** der Klasse zuständig

Annotation	Instanziierung
@Singleton	Einmal pro Programmdurchlauf
@Prototype	Bei jeder Injection
@Context	Während Programmstart
@RequestScope	Bei jedem Request

```
@Singleton  
public class MissionService {
```

Constructor Injection

- Framework injiziert Dependency über Constructor
- Sicher gegenüber NullPointern
- Constructor kann auch normal aufgerufen werden
 - Sinnvoll im Test

➔ **Von Micronaut empfohlen**

```
@Controller("/missions")
public class MissionController {

    private final MissionService missionService;

    public MissionController(MissionService missionService) {
        this.missionService = missionService;
    }
}
```

Field Injection

- Feld wird mit *@Inject* Annotation markiert
- darf nicht final sein
- Gefahr von NullPointerException
- Probleme mit *private*
 - benötigt Reflection

→ **Nicht Empfohlen**

```
@Controller(🌐✓"/missions")
public class MissionController {

    @Inject
    MissionService missionService;
```

Method Parameter Injection

- Setter mit *@Inject* Annotation
- Darf nicht *final* sein
- Außer in **Sonderfällen nicht empfohlen**
 - Optional
 - Varianten
 - Re-Injection

```
@Controller("/missions")
public class MissionController {

    private MissionService missionService;

    @Inject
    void setMissionService(MissionService missionService) {
        this.missionService = missionService;
    }
}
```

Priorität von Beans

- Mehrere mögliche Beans stehen zur Verfügung, Micronaut kann sich nicht entscheiden
- *@Primary*
 - wähle immer die markierte Bean
- *@Required()*
 - Instanziere Bean nur bei Vorbedingung
 - z.B. bei Konfiguration, andere Bean ist vorhanden
- *@Named()*
 - Bei der Injection

```
@Requires(beans = MissionLogRepository.class)
@Requires(property = "game-server.base-url")
public class MissionService {
```

```
public MissionService(@Named("InMemoryMissionRepository") MissionRepository missionRepository) {
    this.missionRepository = missionRepository;
}
```

Optionale Abhängigkeiten

- Micronaut wirft keine Fehler bei **optionalen** fehlenden Beans

```
public MissionController(@Nullable MissionService missionService) {  
    this.missionService = missionService;  
}
```

```
public MissionController(Optional<MissionService> missionService) {  
    this.missionService = missionService;  
}
```


Factory

- Um externe Abhängigkeiten zur Bean zu machen
- Factory Klasse mit `@Factory`
- Function mit Bean Annotation erstellt Bean

```
@Factory
class AWSFactory {

    @Singleton
    AWS aws() {
        return new AWS();
    }
}

class AWS {}
```

Aufgabe 3 Modular Retrofitting (Dependency Injection)

Kommandeur, wir haben über eine neue Methode kennengelernt, die es uns erlaubt, unsere Fabriken modularer zu gestalten! Wir denken, dass diese auch für dich sinnvoll sein können. Wir nennen es **Dependency Injection**. Damit können wir Systeme tauschen, ohne die ganze Station umzubauen.

Cheat Sheet

- A. Baue den Constructor in *MissionController* und *MissionService* so um, dass die Dependencies injiziert werden
 - Denke dabei daran, die Dependencies zu Beans zu machen
- B. Erstelle ein Interface *MissionLogRepository* von dem *InMemoryMissionLogRepository* und *DatabaseMissionLogRepository* erben
- C. Annotiere die beiden Klassen so, dass wenn die Bean *JPAMissionLogRepository* vorhanden ist das *DatabaseMissionLogRepository* genutzt wird, ansonsten soll *InMemoryMissionLogRepository* genutzt werden
- D. Probiere aus, ob du Unterschiede merkst, wenn *MissionRng* mit *@Prototype* oder mit *@RequestScope* annotiert ist

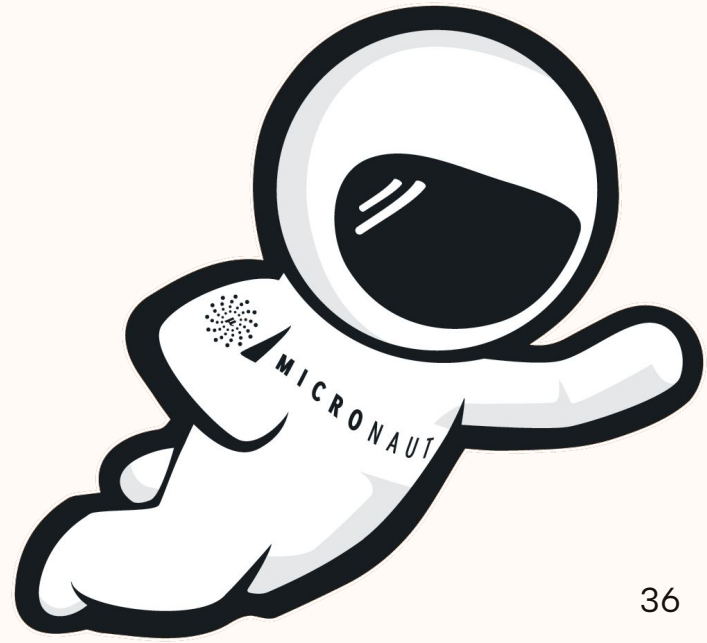
```
@Singleton  
public class MissionService {
```

```
@Controller("@*/missions")  
public class MissionController {  
  
    private final MissionService missionService;  
  
    public MissionController(MissionService missionService) {  
        this.missionService = missionService;  
    }  
}
```

```
@Requires(bean = MissionLogRepository.class)  
@Requires(property = "game-server.base-url")  
public class MissionService {
```

```
@Singleton  
@Primary  
public class InMemoryMissionLogRepository
```

Pause 15 Minuten



REST Clients

Micronaut REST Clients

- Interfaces statt Implementierung
- REST Clients sind häufig Boilerplate
- Type-Safety und Refactoring-freundlich
- Non-blocking IO (Netty) (!= Non-blocking Code!)

```
@Client(id = "game-server")
public interface GameServerClient {

    @Get("/one/endpoint/refresh")
    void refreshSomething();
}
```

```
@Generated
class GameServerClient$Intercepted implements GameServerClient, Introduced {
    private final Interceptor[][] $interceptors;
    private final ExecutableMethod[] $proxyMethods;

    public RadarScanResponseDto scan(String token, String shipId) {
        return (RadarScanResponseDto)(new MethodInterceptorChain((Interceptor[])this.$interceptors[0], this, this.$proxyMethods[0], null));
    }

    public void setCourse(String token, SetCourseRequestDto req) {
        (new MethodInterceptorChain((Interceptor[])this.$interceptors[1], this, this.$proxyMethods[1], null)).invoke(this, req);
    }

    public List<ShipStatusDto> myShips(String token) {
        return (List)(new MethodInterceptorChain((Interceptor[])this.$interceptors[2], this, this.$proxyMethods[2], null));
    }

    public ShipStatusDto registerShip(String token, RegisterShipRequestDto request) {
        return (ShipStatusDto)(new MethodInterceptorChain((Interceptor[])this.$interceptors[3], this, this.$proxyMethods[3], null));
    }
}
```

Micronaut REST Clients

- Jede HTTP Methode hat ihr @Interface Äquivalent
 - @Get, @Post, @Delete, @Head, ...
- Placeholder werden mit {derName} angegeben
 - Mappt direkt zu Parameter **derName**
- @Body mappt direkt zu POJO
 - POJO muss Introspected sein und Getter/Setter oder AllArgsConstr haben
- Return Types erlauben verschiedene Varianten
 - POJO, Optional<T>, HttpResponse<T>, Reactive Types
 - POJO und HttpResponse sind blocking (wartet auf Ergebnis)
 - Reactive Responses mit Publisher, Mono, Flux, CompletableFuture
 - **Out of Scope!**
- @CircuitBreaker erlaubt Retries bei Fehlern

```
@Fallback
@Slf4j
public class GameServerClientFallback implements GameServerClient {

    @Override
    public RegisterShipResponseDto registerTeam(RegisterTeamRequestDto request) {
        log.warn(msg: "Fallback: Register Team failed.");
        return null;
    }
}
```

Aufgabe 3.5 Take Flight

Kommandeur! Unsere Kommunikationsoffiziere haben eine beunruhigende Entdeckung gemacht.

Letzte Woche ist unser Hauptkommunikationssatellit ausgefallen. Für genau 47 Sekunden. In diesen 47 Sekunden hat unsere Flotte ****23 Befehle**** an den Satelliten für unsere Raumschiffe geschickt – und ALLE sind ins Leere gelaufen.

Helft uns und implementiert Aufgabe 3!

AOP

Aspect Oriented Programming (AOP)

Was bedeuten die Annotationen eigentlich?

- Ziel: Trennen von **Logik** und **Boilerplate / Plumbing**

```
void transferMoney() {  
    log.info(msg: "Starting.");  
  
    try(DataBaseConnector db = new DataBaseConnector(url: "jdbc://postgresql:5432/myDB")) {  
        db.beginTransaction();  
  
        // ... actual logic ...  
  
        db.commit();  
    } catch (Exception e) {  
        rollback();  
        log.error(msg: "Transaction failed!", e);  
    }  
  
    log.info(msg: "Transaction successful.");  
}
```

Aspect Oriented Programming (AOP)

```
@Logged  
@Transactional  
@ExceptionHandler  
void transferMoney() {  
    // ... actual logic ...  
}
```

Aspect Oriented Programming (AOP)

- Automatische Durchführung von wiederholdenden Patterns
 - Logging
 - Metrics
 - Transaktionen für Datenbanken
 - Authentifizierung
- **Annotations:** Metadaten (Sticky Note), markieren die Methode / Klasse
- **Interceptor: Was** soll ausgeführt werden?
- **Pointcut: Wo** soll der Interceptor ausgeführt werden?
 - z. B. nur an Methoden, nur an Konstruktoren*, nur auf Feldern, nur auf Klassen, während Buildtime oder Runtime
 - **Auch:** Kombinationen möglich!
- **Advice: Wann** soll es ausgeführt werden?
 - Vor der Methode, nach der Methode, AroundInvoke

*schwierig, aber möglich ohne Micronaut
Benötigt z. B. AspectJ oder ByteBuddy 44

Aspect Oriented Programming (AOP)

- @Retention: Bestimmt Lebenszeit
 - SOURCE: Wird vom Compiler weggeworfen
 - CLASS (Default): Wird in .class eingefügt, aber JVM ignoriert es
 - RUNTIME: Wird von der JVM in den Speicher geladen
- @Target: Wo kann es genutzt werden?
 - Üblich: METHOD, TYPE (= Klassen), PARAMETER, CONSTRUCTOR
 - Annotieren mit CONSTRUCTOR möglich
 - Aber Interceptor mit **@AroundConstruct!**
 - Advanced: TYPE_USE
 - Kann für Casts und Generics benutzt werden
 - List<@NotNull String>
 - (@NotNull MyType) x;
- Specific overrides general → Methode gewinnt über Klasse
- Verschachtelung möglich!
- Interceptors sind normale Singleton Beans! → Injection möglich

Aspect Oriented Programming (AOP)

Ein Beispiel:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface LogMe { } // Just a marker!
```

Frage:

Da Methodenaufrufe in Java
"direct memory calls" sind, wie kann
unser Interceptor zwischen
Aufrufer und Methode gelangen?

```
@Singleton
@InterceptorBean(LogMe.class)
public class LogMeInterceptor implements MethodInterceptor<Object, Object> {

    @Override
    public Object intercept(MethodInvocationContext<Object, Object> context) {

        String name = context.getMethodName();
        var args = context.getParameterValues();

        log.debug(format: "[LOG] Enter {}({})", name, args);

        try {
            Object result = context.proceed();
            log.debug(format: "[LOG] Exit {}()", name);
            return result;
        } catch (Throwable t) {
            log.error(format: "[LOG] Exception in {}(): {}", name, t.toString());
            throw t;
        }
    }
}
```

Dependency Injection under the Hood

Wie funktioniert die Dependency Injection eigentlich?

Historisch: WildFly

- Applikationsstart: Heavy Deployment Scan
- Server durchsucht das gesamte Archiv (.war / .ear)
- Scannt WEB-INF/classes und alle JARs in WEB-INF/lib
- Aufbau des Weld-Metamodells im Arbeitsspeicher (Memory-Intensiv!)
- **Problem:** $O(n)$ Komplexität
 - Je mehr Klassen/Abhängigkeiten, desto länger dauert das Deployment (oft Minuten).
- Aufrufe an die Bean werden **vom Weld-Proxy** verarbeitet!

Probleme:

- Deployment Hell: UnsatisfiedDependencyException fliegt erst zur Laufzeit
- Speicherverbrauch: Alle Klassen im RAM, auch ungenutzte
- Ewige Startup-Time (nicht nutzbar für die **Cloud**)
- Overhead: Jeder Methodenaufruf muss erst den aktiven Context prüfen (Lookup-Kosten).

Spring: Convention over Configuration

Wie funktioniert die Dependency Injection eigentlich?

Historisch: Spring Boot

- Applikationsstart: Classpath Scan (classpath*:com/myapp/**/*.class)
- Scan mit Java Reflection für Metamodell
- **Problem:** Dauer skaliert linear mit Codebase-Größe!
- Beans werden in dynamische Proxies "eingepackt" (CGLIB / JDKProxy)
- Aufrufe an die Bean werden **vom Proxy** verarbeitet!
 - myService.doSomething() → CglibProxy.doSomething()
 - Dieser prüft dann Annotationen

Probleme:

- Lange Stacktraces
- Schwierig zu debuggen
- Kostet sehr viel Memory
- Lange Startzeit
- Funktionsaufrufe sind langsamer

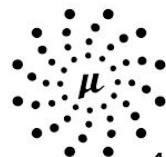


Micronaut: Natively cloud-native Development

- Statt zur Runtime zu prüfen: Bei Compiletime berechnen!
 - Ahead-of-Time Modell (AOT) mit Java Annotation Processings Tool (APT)
 - Während Compilation: Neue Dateien generieren
 - MyClass\$Definition.class, MyClass\$Intercepted.class
 - **Vorteil:** Normale Java-Klassen, sofort verfügbar
 - Falls Injection notwendig: Unterklasse wird erstellt
 - Überschreibt Methoden
 - Keine Reflection notwendig
 - Reflection durch statische **BeanIntrospection** (**@Introspected**) ersetzt
- **Nichts darf final sein!**

Vorteile:

- Sehr einfach zu debuggen (normaler Java-Code, einfach lesbar)
 - Probiert das später gerne mal aus!
- Kostet wenig Memory
- Kurze Startzeit (Super für Cloud)
- Funktionsaufrufe sind gleich schnell



Quarkus: Supersonic Subatomic Java

- Build-Time Augmentation Modell
- "Closed World" Annahme
- Optimierte den Bytecode aggressiv
 - Build-Time CDI Container (Arc)
 - Jandex (Java Annotation Index)
- Während mvn package:
 - Analysiert Applikation, findet Injection Points
 - Entfernt "Dead Code", Gizmo schreibt Startup-Code direkt in Bytecode
- Gleicher Hersteller wie WildFly

Vorteile:

- Sehr schnell, quasi kein extra Class-Loading notwendig
- Sehr gute Integration mit GraalVM

Probleme:

- Umständlich zu debuggen
- Viele Bibliotheken funktionieren nicht nativ (quarkiverse)
- Stacktraces sind oft nichtssagend



Aufgabe 4 AOP

- a) Implementiert das `@interface` für die Klasse `de.riversroses.infra.logging.Logged`. Dieses soll ein `AroundInvoked` Interface sein für Methoden zur Runtime.
- b) Implementiert nun den `Interceptor` `de.riversroses.infra.logging.LoggedInterceptor`. In dieser Klasse sind einige Kommentare von unseren vorherigen Engineers hinterlassen worden.
- c) Fügt die Annotation `de.riversroses.missions.rest.MissionControllers` Methoden hinzu! Prüft mit `curl`, ob die Log-Nachrichten tatsächlich ankommen.
- d) Wie könntet ihr die Annotation erweitern, so dass diese auf Klassen funktioniert? Tipp: Schaut auf den Folien. Implementiert dies!
- e) Ersetzt eure Method-Level Annotationen nun durch eine Class-Level Annotation und versucht das `curl` erneut!

Quizz: Was ist das?

(?:[a-z0-9!#\$%&'*/+=?^_`{|}~-]+(?:\. [a-z0-9!#\$%&'\*/+=?^\_`{|}~-]+)*|"(?:[\x01-\x08
 \x0b\x0c\x0e-\x1f\x21\x23-\x5b\x5d-\x7f]|\\[\x01-\x09\x0b\x0c\x0e-\x7f])*"
)@(?: (?:[a-z0-9](?:[a-z0-9-]*[a-z0-9])?\.)+[a-z0-9](?:[a-z0-9-]*[a-z0-9])?|\b(?:25[
 0-5]|2[0-4][0-9]|01)?[0-9][0-9]?)\.){3}(?:25[0-5]|2[0-4][0-9]|01)?[0-9][0-9]?|[a-z0-9
 -]*[a-z0-9]:(?:[\x01-\x08\x0b\x0c\x0e-\x1f\x21-\x5a\x53-\x7f]|\\[\x01-\x09\x0
 b\x0c\x0e-\x7f])+)\b)

Validation



Was ist Data Validation?

- Automatische Prüfungen, ob Daten bestimmte Regeln erfüllen
 - Ist der String eine E-Mail?
 - Sind alle Datenfelder gefüllt?
 - ...
- Sorgt für:
 - Sicherheit
 - Datenintegrität
 - Businesslogic & Compliance
 - UX



Data Validation in Micronaut

- `jakarta.validation`
- Controller benötigt “@Validated” für Kompilierung
- Annotation bei Variablen-Deklarationen
- Wir reden auch von: “Constraints”
- Mögliche Validation Annotationen:
 - `@AssertFalse`
 - `@DecimalMax(“int”)`
 - `@Email`
 - `@NotEmpty`
 - `@NotNull`
 - ...

→ `jakarta.validation` Doku

```
import io.micronaut.http.HttpResponse;
import io.micronaut.http.annotation.Controller;
import io.micronaut.http.annotation.Get;
import io.micronaut.validation.Validated;

import jakarta.validation.constraints.NotBlank;
import java.util.Collections;

@Validated // (1)
@Controller("/email")
public class EmailController {

    @Get("/send")
    public HttpResponse send(@NotBlank String recipient, // (2)
                             @NotBlank String subject) { // (2)
        return HttpResponse.ok(Collections.singletonMap("msg", "OK"));
    }
}
```

Data Validation mit DTOs (1/2)

- Teilung der Endpunkten und Daten in Form von DTOs
"Data Transfer Objekts"
- Annotationen für DTOs:
 - `@Introspcted` // Erlaubt Micronaut den Zugriff auf Constraints
 - (Immer wenn Datavalidation genutzt wird nutzen)
 - `@Serdeable` // Erlaubt JSON-Konvertierung
 - (Immer, wenn serialisiert werden soll)
 - Incl. Introsepcted

```
@Introspcted // Erlaubt Micronaut den Zugriff auf Constraints
public class EmailRequest {

    @NotBlank(message = "Empfänger darf nicht leer sein")
    @Email(message = "Ungültiges E-Mail-Format")
    private String recipient;

    @Size(min = 5, max = 100)
    private String subject;

    // Getter & Setter oder Record-Constructor
}
```


Data Validation mit DTOs (2/2)

- Controller mit `@Validated` annotieren → Macht Compiler darauf aufmerksam, dass hier Validation geprüft werden muss
- Bei Nutzung der DTO als Parameter:
- `@Valid` auf Klasse / Methodenparameter → die Felder der Klasse müssen valide sein, Validation geschieht automatisch

```
@Validated // Aktiviert die Validierungs-AOP
@Controller("/email")
public class EmailController {

    @Post("/send")
    public HttpResponseMessage<?> send(@Body @Valid EmailRequest request) {
        // Wenn der Code hier ankommt, ist das DTO bereits valide!
        return HttpResponseMessage.ok();
    }
}
```

Validation Groups (1/2)

- Feinere Validation Unterteilungen in Objekten
- Standardmäßig verwendete Klasse: "Default"
- Definition einer Validation Group durch Interface (1)
- Extends → Impliziert in Gruppe
- Zuordnung einer Validation zu einer Gruppe (2)



```
import jakarta.validation.groups.Default;  
  
public interface FinalValidation extends Default {} // (1)
```

```
@NotBlank(groups = FinalValidation.class) // (2)  
String recipient;
```

Validation Groups (2/2)

- @Validated
 - "Default" Validation Group (1)
- @Validated(groups =[Group]) (2)
 - Methode/Controller Validiert nach [Group]
- @Valid [Klasse]
 - Führt "aktive" Validierung aus

```
@Validated // (1)
@Controller("/email")
public class EmailController {

    @Post("/createDraft")
    public HttpResponse createDraft(@Body @Valid Email email) { // (2)
        return HttpResponse.ok(Collections.singletonMap("msg", "OK"));
    }

    @Post("/send")
    @Validated(groups = FinalValidation.class) // (3)
    public HttpResponse send(@Body @Valid Email email) { // (4)
        return HttpResponse.ok(Collections.singletonMap("msg", "OK"));
    }
}
```

Operation: Data-Airlock

Komander(m/w/d)! Weltraumpiraten versuchen, uns mit korrupten Datenpaketen zu sabotieren. Wir müssen unsere Schotten dicht machen! Bitte beschützt uns!

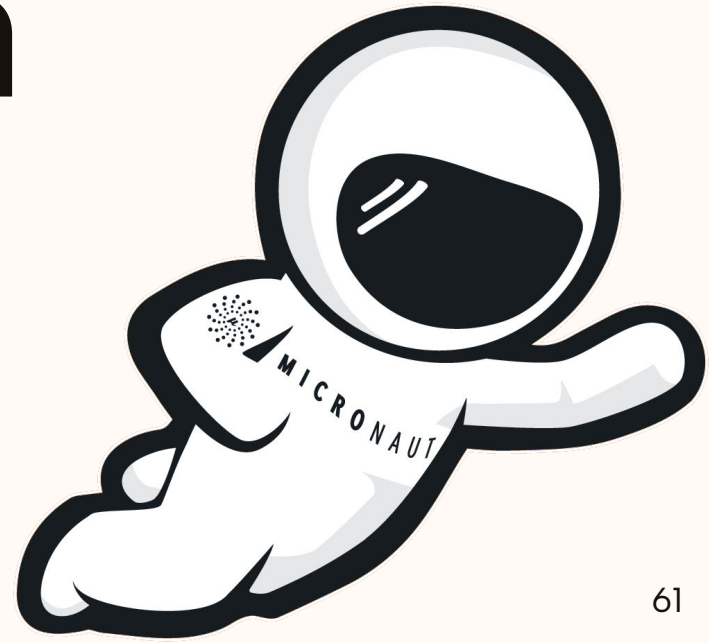
Eure Aufgabe:

Implementiert einen Error Handler, der Validierungsfehler abfängt und die Piraten mit maximaler Verwirrung bestraft.

- A. Nutzt die `@Error`-Annotation direkt in eurem MissionController. Ziel ist es, den Standard-Validierungsfehler (der durch `@Valid` ausgelöst wird) abzufangen und die Piraten abzuweisen.
- B. Fügt eine Methode im MissionController hinzu, die auf die `ConstraintViolationException` reagiert.
- C. Markiert diese Methode mit der `@Error`-Annotation. Nutzt `HttpResponse.status(HttpStatus.I_AM_A_TEAPOT)`
- D. Versucht eine ungültige `MissionCompletion` zu senden - es sollte nun mit 418 I'm a teapot antworten. Tipp: `curl -i localhost:8081/missions` Das `-i` zeigt hier minimale Zusatzinformationen an, inklusive dem Status-Code.



Mittagspause 45 Minuten



Errorhandling

sometimes bad things happen.... EVEN IN SPACE!

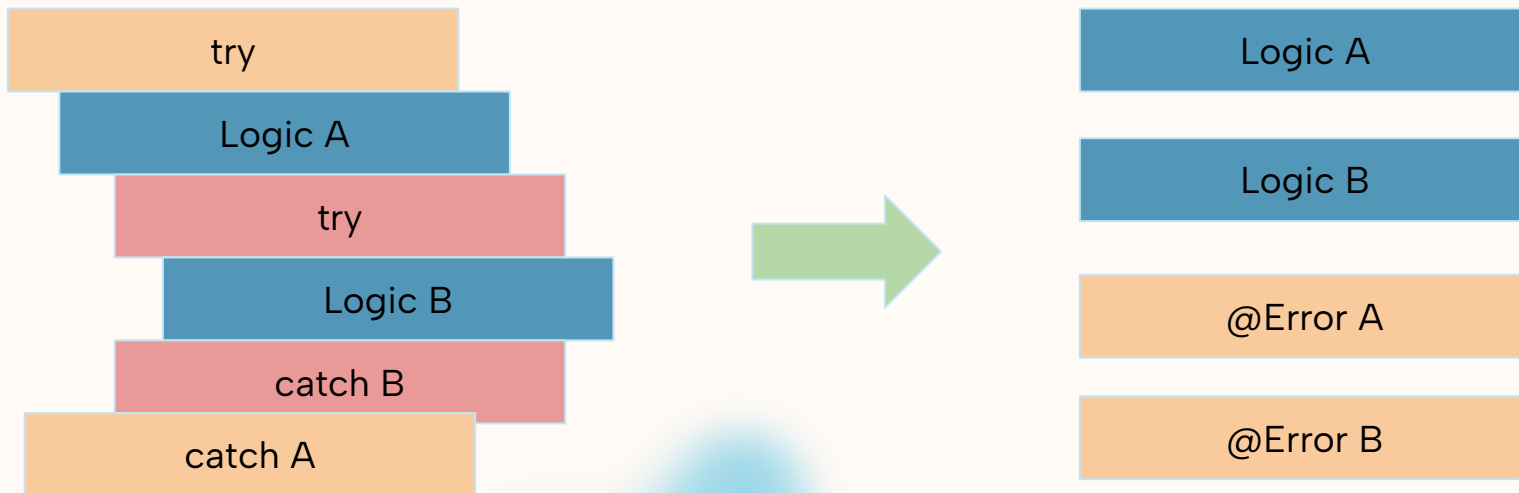
Errors

- Fehler sind in (verteilten) Systemen unvermeidbar [Murphy1977]
- Ziel ist, konsistent mit Fehlern umzugehen
 - Keine Abstürze
 - Klare Rückmeldung



Errors in Micronaut

- Trennung von Business-Logik und Fehlerbehandlung
- Deklarativ, statt verschachtelte Try-Catch-Blocks



@Error Annotation

- Methoden innerhalb eines @Controllers, die für spezifische Fehler zuständig sind
- 2 Verschiedene Scopes:
 - Local – Nur für diesen Controller zuständig
 - Global – Für die ganze Applikation (Keine Überdeckung möglich)
- Annotation:
 - @Error(
 - exception = [Exceptionclass] ||
 - status = [HTTP Status code] ||
 - global = [boolean])

@Error Annotation Beispiel

HTTP Statuscode

```
@Error(status = HttpStatus.NOT_FOUND)
public HttpResponse<JsonError> notFound(HttpRequest request) {
```

Exceptionclass (implizit durch AOT)

```
@Error
public HttpResponse<JsonError> jsonError(HttpRequest request, JsonSyntaxException e) {
```

```
=> @Error(exception = JsonSyntaxException.class)
```

ExceptionHandler

- Separate Klasse implementiert ExceptionHandler<T,R>
- Methoden überschreiben mit @Override standardhandler
- Wenn Logik zu komplex für Controller
- Wenn Handler über mehrere Services geteilt werden soll
- Wichtig:
 - Klasse sollte als Singleton annotiert sein

```
@Singleton
@Produces
public class CustomExceptionHandler
    implements ExceptionHandler<CustomException, HttpResponseMessage> {

    @Override
    public HttpResponseMessage handle(HttpRequest request, CustomException exception) {
        return HttpResponseMessage.badRequest("Spezifischer Fehler: " + exception.getMessage())
    }
}
```



Validation Exceptions

- Normales Verhalten:
 - 400er Code + Validation Message
- Eigenen ExceptionHandler für Constraints mit
 - `"@Replaces(ConstraintExceptionHandler.class)"`
- Validation Errors werfen:
 - `"jakarta.validation.ConstraintViolationException"`

```
{
  "_embedded": {
    "errors": [
      {
        "message": "Dieses Feld darf nicht null sein"
      }
    ]
  },
  "message": "Bad Request"
}
```

```
@Replaces(ConstraintExceptionHandler.class)
public class ViolationHandler implements ExceptionHandler<ConstraintViolationException, HttpResponse<?>> {
```

Aufgabe 5 Operation Teapot!

Eure Aufgabe: Ersetzt eure manuellen if/else-Prüfungen durch Micronaut Bean Validation.

- a) Nutzt `@NotNull`, `@NotBlank`, `@Min(1)` und `@Positive` in euren DTOs.
- b) Aktiviert die Validierung in eurem Controller mit der `@Valid` Annotation am Parameter. Ersetzt eure if/else Prüfungen durch Micronaut Validation. Es sollten keine if/else-Anweisungen mehr im Controller vorhanden sein.
- c) Prüft nun eine ungültige `MissionCompletion` zu senden. Euer Planet sollte mit einem 400 Bad Request antworten, noch bevor euer Code ausgeführt wird!

Testen

Warum testen wir unsere Services?

- Änderungen brechen schnell Dinge
- Fehler spät zu finden ist teuer
- Microservices müssen stabil sein
- Tests geben Sicherheit

Welche Testarten betrachten wir?

- Smoke-Test => Liveness
- Integration / REST-Tests => API-Verhalten
- Unit-Tests => Logik

Heute im Workshop: Fokus liegt auf REST-Tests

Unit-Tests

- Testen einzelne Methoden
- Kein HTTP
- Kein Application Context
- assertEquals & assertTrue , ...

```
@Test
void addTest() {
    assertEquals(4, 2 + 2);
}
```

Testen in Micronaut

Intermission: J-UNIT

assertEquals

assertTrue

```
@Test
void addTest() {
    assertEquals(4, 2 + 2);
}
```

Testen in Micronaut – Grundidee

- Echter Application Context
- Embedded HTTP-Server
- Wenig Setup
- Sehr schnelle Tests

REST-Test: Hello Endpoint

```
@MicronautTest
class HelloControllerTest {

    @Inject
    HttpClient client;

    @Test
    void testHello() {
        String response = client.toBlocking()
            .retrieve("/hello");

        assertEquals("Hello World", response);
    }
}
```

Testen in Micronaut

Validation testen

```
@Test
void invalidRequestShouldFail() {
    HttpRequest<String> request =
        HttpRequest.POST("/user", "");

    assertThrows(HttpClientResponseException.class, () ->
        client.toBlocking().exchange(request)
    );
}
```

Testing der API-Endpunkte in Micronaut

Ziel:

- Prüfen, ob HTTP-Endpunkte korrekt funktionieren
- Business-Logik + HTTP-Schicht zusammen testen
- Validierung & Statuscodes prüfen

Beispiel aus unserem Projekt:

```
GET /missions  
POST /missions/complete
```

Besonderheit bei Micronaut Tests

In Micronaut testen wir nicht nur Methoden – wir starten einen **echten Embedded Server**.

Dafür brauchen wir:

```
@MicronautTest
```

Was passiert dadurch?

- ApplicationContext wird gestartet
- Embedded HTTP Server läuft
- Beans werden injected
- Test läuft im Environment: `test`

Micronaut startet wirklich die komplette Anwendung.

Dependency Injection im Test

In Micronaut können wir im Test ganz normal injecten:

```
@Inject  
MissionLogRepository repo;
```


Das Besondere: HttpClient im Test

In klassischen Unit-Tests ruft man Controller-Methoden direkt auf.

In Micronaut machen wir das anders:

```
@Inject
@Client("/")
HttpClient client;
```

Was bedeutet das?

- `@Client("/")` erzeugt einen HTTP-Client
- Der Client spricht mit dem Embedded Server
- Wir testen echte HTTP-Requests

Das ist ein Integrationstest, kein Unit-Test.

Beispiel: GET Test

```
MissionPayloadDto payload = client.toBlocking()  
    .retrieve(HttpRequest.GET("/missions"), MissionPayloadDto.class);
```

Was passiert hier?

1. HTTP GET wird an /missions gesendet
2. Controller wird aufgerufen
3. HttpResponse wird zurückgegeben

Dann prüfen wir:

```
assertEquals("Seeded mission", payload.description);
```

Arrange / Act / Assert Pattern

Micronaut-Tests folgen normalem JUnit-Stil:

Arrange

```
repo.save(seeded);
```

Act

```
client.toBlocking().retrieve(...)
```

Assert

```
assertEquals(...)
```

Testing von POST + Statuscodes

Beispiel:

```
var response = client.toBlocking()  
    .exchange(HttpRequest.POST("/missions/complete", completion));
```

Warum `.exchange()` statt `.retrieve()`?

- `.retrieve()` → nur Body
- `.exchange()` → komplette Response (inkl. Status)

Dann prüfen wir:

```
assertEquals(HttpStatus.BAD_REQUEST, ex.getStatus());
```

Testing von Validation (400)

Wenn Validation fehlschlägt:

```
HttpClientResponseException ex = assertThrows(  
    HttpClientResponseException.class,  
    () -> client.toBlocking().exchange(...)  
);
```

Warum Exception?

Weil Micronaut bei 400 eine Exception wirft.

Dann prüfen wir:

```
assertEquals(HttpStatus.BAD_REQUEST, ex.getStatus());
```

Recap Testing

1- Klassenkopf & Setup

```
@MicronautTest  
class MissionControllerTest {
```

`@MicronautTest` startet die komplette **Micronaut-Anwendung im Testmodus**. Es wird:

- Dependency Injection aktiviert
- HTTP-Server gestartet
- H2-Testdatenbank gestartet
- Validation aktiviert

2- Dependency Injection

```
@Inject  
@Client("/")  
HttpClient client;
```

`@Inject` => Micronaut gibt uns automatisch ein Objekt
`@Client("/")` => HTTP-Client für unsere eigene Anwendung
Damit können wir echte HTTP-Requests an unsere Controller senden.

```
@Inject  
MissionLogRepository repo;
```

Hier bekommen wir das Repository direkt injiziert.

- Testdaten vorbereiten
- Datenbankzustand nach dem Request prüfen

Recap Testing

3- Datenbank vor jedem Test leeren

```
@BeforeEach
void cleanDb() {
    repo.deleteAll();
}
```

Test: Mission existiert bereits

```
void getMissions_returnsMissionsFromRepository()
```

a) Arrange

```
MissionLog seeded = new MissionLog("mission-123", "Seeded mission", 10.0, 20.0);
repo.save(seeded);
```

Wir legen manuell eine Mission in die DB
Status ist automatisch PENDING

b) Act

```
MissionPayloadDto payload = client.toBlocking()
    .retrieve(HttpRequest.GET("/missions"), MissionPayloadDto.class);
```

1. HTTP GET auf `/missions`
2. Controller wird aufgerufen
3. Service wird aufgerufen
4. Repository wird abgefragt
5. Antwort wird als `MissionPayloadDto` zurückgegeben

`toBlocking()` macht den asynchronen Client synchron (für Tests einfacher).

c) Assert

```
assertNotNull(payload);
assertEquals("Seeded mission", payload.description);
```

Wir prüfen: Response ist da? Werte stimmen?

Aufgabe 7 : Testing

Schreiben Sie **4 Integrationstests** für den `MissionController` mit:

1- GET /missions – Mission existiert

- Speichern Sie vor dem Request eine `PENDING` Mission im Repository.
- Senden Sie einen GET-Request an `/missions`.
- Prüfen Sie:
 - Die zurückgegebene Mission entspricht der gespeicherten.
 - Es existiert weiterhin genau **eine** `PENDING`-Mission

2- GET /missions – Keine Mission vorhanden

- Repository ist leer.
- Senden Sie einen GET-Request an `/missions`.
- Prüfen Sie:
 - Eine Mission wird generiert.
 - Genau **eine** `PENDING`-Mission wurde gespeichert.

Aufgabe 7 : Testing

Schreiben Sie **4 Integrationstests** für den `MissionController` mit:

3- POST /missions/complete – Erfolgreich

- Speichern Sie eine `PENDING` Mission.
- Senden Sie einen gültigen POST-Request.
- Prüfen Sie:
 - HTTP Status = `200 OK`
 - Status der Mission ist `COMPLETED`
 - Completion-Felder sind gesetzt

4- POST /missions/complete – Ungültiger Body

- Senden Sie einen POST-Request mit leerem Body.
- Prüfen Sie:
 - `400 BAD_REQUEST`
 - Exception wird korrekt geworfen.

Recap

Open API

Recap OpenAPI?

Was ist OpenAPI?

- OpenAPI ist Standard zur Beschreibung von REST-APIs
- Beschreibt:
 - Endpunkte
 - Parameter
 - Responses
- Grundlage für Tools wie Swagger UI

OpenAPI $\neq$ Dokumentation

OpenAPI = Spezifikation

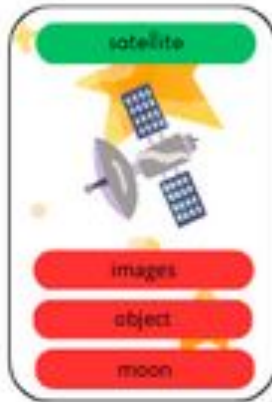
Wer hat schon Zeit zu tippen..

Von Code zu OpenAPI

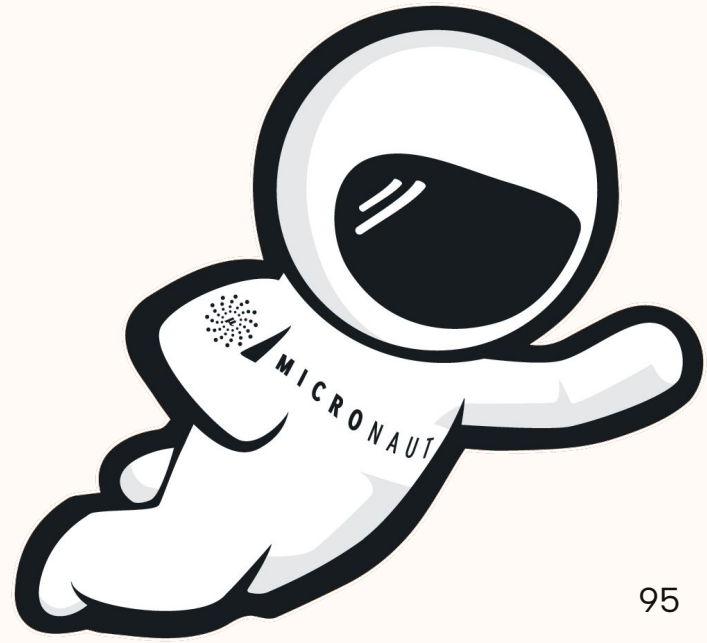
- **Ziel:** Robustheit, indem der Code sich möglichst selbst spezifiziert
- Automatische Generierung von OpenAPI Doku

```
outer:  
  static-resources:  
    swagger:  
      paths: classpath:META-INF/swagger  
      mapping: /swagger/**  
    swagger-ui:  
      paths: classpath:META-INF/swagger/views/swagger-ui  
      mapping: /swagger-ui/**  
  default:  
    enabled: true  
    mapping: /**  
    paths: classpath:public
```

Taboo



Pause 15 Minuten



Konfiguration

@Jules TODO

Datenzugriff

GraalVM

GraalVM

- fortschrittliche JDK
- AOT Kompilierung
- Natives Image
- bessere Performance, dafür höhere Compilezeit
- Synergien mit GraalVM
 - AOT



```
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 08:40 min
[INFO] Finished at: 2026-02-11T08:11:31+01:00
[INFO] -----
```

Normale Ausführung
Speicher: 350 MB

GraalVM
Speicher: 36 MB

Zusammenfassung



Solltet ihr Micronaut verwenden?

Vorteile

- Schnell
- Wenig Speicherverbrauch
- Serverless/Cloud-Native
- Viele Integrationen

Nachteile

- Im Vergleich zu Spring
 - Kleinere Community
 - Weniger Integrations
 - Weniger Stabil
 - Weniger Entwickler
 - Weniger Tutorials

**Danke für Eure
Aufmerksamkeit!**